

KOMPLETNO OBJAŠNJENJE PROJEKTA: EdgeAI RPi

Docker

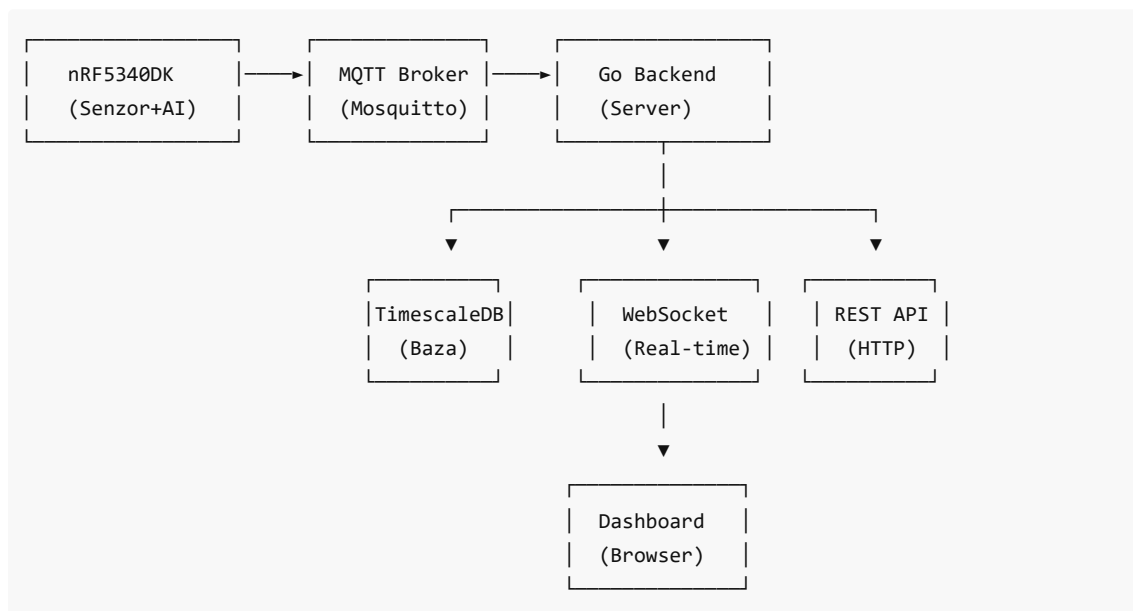
PREGLED PROJEKTA

Što ovaj projekt radi?

Ovaj projekt je **sustav za prediktivno održavanje** industrijskih motora. Zamislite tvornicu s motorima - ovaj sustav:

1. **Prima podatke** sa senzora (vibracije motora)
2. **Detektira kvarove** pomoću AI modela na mikrokontroleru
3. **Sprema podatke** u bazu podataka
4. **Prikazuje rezultate** na web dashboard-u u realnom vremenu

Arhitektura sustava



DOCKER I KONTEJNERIZACIJA

Što je Docker?

Docker je tehnologija koja omogućuje **pakiranje aplikacija u "kontejnere"**. Kontejner je kao mali virtualni stroj, ali puno lakši i brži.

Analogija: Zamislite shipping container (kontejner za transport). Sve stavite unutra i možete ga prevoziti bilo gdje - na brod, kamion, vlak. Docker kontejner je isto - pakira vašu aplikaciju sa svime što joj treba i možete je pokrenuti bilo gdje.

Zašto Docker?

1. **Konzistentnost** - "Works on my machine" problem nestaje
2. **Izolacija** - aplikacije ne utječu jedna na drugu

3. Jednostavno deployanje - jedna naredba pokreće sve

DATOTEKA: docker-compose.yml

Docker Compose omogućuje definiranje i pokretanje **više kontejnera odjednom**. Umjesto da ručno pokrećete svaki kontejner, napišete jednu datoteku i pokrenete sve.

```
services:
```

Objašnjenje: `services:` označava početak definicije svih servisa (kontejnera) u aplikaciji.

Servis: Baza podataka (db)

```
db:
  image: timescale/timescaledb:2.14.2-pg14
```

Objašnjenje:

- `db:` - naziv servisa (možemo ga koristiti kao hostname unutar Docker mreže)
- `image:` - koristi gotovu sliku s Docker Hub-a
- `timescale/timescaledb:2.14.2-pg14` - TimescaleDB verzija 2.14.2 bazirana na PostgreSQL 14

Što je TimescaleDB?

TimescaleDB je **proširenje PostgreSQL-a** optimizirano za **vremenske serije podataka** (time-series data). Savršeno za senzorske podatke jer:

- Automatski particionira podatke po vremenu
- Brži upiti na velike količine podataka
- Kompresija starih podataka

```
container_name: edgeai-db
```

Objašnjenje: Eksplicitno imenuje kontejner. Bez ovoga Docker bi generirao nasumično ime poput `projekt_db_1`.

```
environment:
  POSTGRES_DB: edgeai
  POSTGRES_USER: edgeai
  POSTGRES_PASSWORD: edgeai
```

Objašnjenje:

- `environment:` - definira varijable okoline unutar kontejnera
- `POSTGRES_DB` - naziv baze podataka koja će se kreirati pri pokretanju
- `POSTGRES_USER` - korisničko ime za pristup bazi
- `POSTGRES_PASSWORD` - lozinka za korisnika

Što su Environment Variables (Varijable Okoline)?

Varijable okoline su **parovi ključ-vrijednost** dostupni aplikaciji za vrijeme izvršavanja. Koriste se za:

- Konfiguraciju bez mijenjanja koda
- Skrivanje osjetljivih podataka (lozinke)
- Različite postavke za development/production

```
volumes:  
  - db-data:/var/lib/postgresql/data  
  - ./db/setup.sql:/docker-entrypoint-initdb.d/setup.sql
```

Objašnjenje:

- `volumes:` - mapira podatke između host računala i kontejnera
- `db-data:/var/lib/postgresql/data` - **named volume** - Docker upravlja lokacijom, podaci opstaju i kada se kontejner obriše
- `./db/setup.sql:/docker-entrypoint-initdb.d/setup.sql` - **bind mount** - kopira našu SQL skriptu u poseban direktorij koji PostgreSQL automatski izvršava pri prvom pokretanju

Što je Volume?

Volume je mehanizam za **trajno pohranjivanje podataka**. Bez volumea, svi podaci u kontejneru bi nestali kad se kontejner obriše.

```
ports:  
  - "5432:5432"
```

Objašnjenje:

- `ports:` - mapira portove između host-a i kontejnera
- `"5432:5432"` - format je `HOST_PORT:CONTAINER_PORT`
- Port 5432 je standardni port za PostgreSQL
- Omogućuje pristup bazi s host računala (npr. za debugging)

Što je Port?

Port je **virtualna "vrata"** kroz koja aplikacije komuniciraju preko mreže. Svaka aplikacija sluša na određenom portu (npr. web serveri tipično na 80 ili 443).

```
healthcheck:  
  test: ["CMD-SHELL", "pg_isready -U edgeai"]  
  interval: 5s  
  timeout: 5s  
  retries: 5
```

Objašnjenje:

- `healthcheck:` - definira kako Docker provjerava je li servis "zdrav"
- `test:` - naredba koja se izvršava za provjeru
- `pg_isready` - PostgreSQL alat koji provjerava može li baza primiti konekcije
- `interval: 5s` - provjera se izvršava svakih 5 sekundi
- `timeout: 5s` - ako nema odgovora unutar 5 sekundi, smatra se neuspjehom
- `retries: 5` - nakon 5 neuspjelih provjera, servis se smatra nezdravim

Servis: MQTT Broker (mqtt)

```
mqtt:
  image: eclipse-mosquitto:2
```

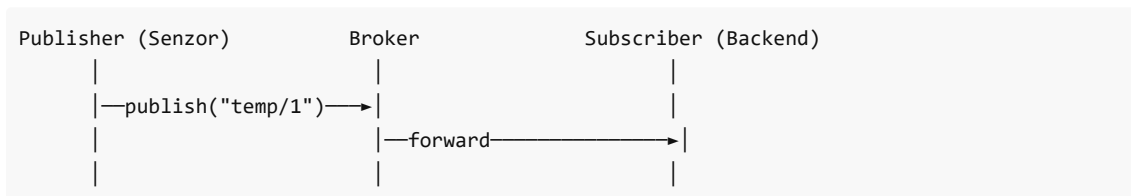
Objašnjenje: Koristi Eclipse Mosquitto verziju 2 - popularan open-source MQTT broker.

Što je MQTT?

MQTT (Message Queuing Telemetry Transport) je **lagani protokol za razmjenu poruka**. Dizajniran za:

- IoT uređaje s ograničenim resursima
- Nestabilne mrežne veze
- Komunikaciju publish/subscribe tipa

Publish/Subscribe Pattern



- **Publisher** objavljuje poruku na određeni **topic** (kanal)
- **Subscriber** se pretplati na topic
- **Broker** prosljeđuje poruke od publishera subscriberima

```
container_name: edgeai-mqtt
ports:
  - "1883:1883"
```

Objašnjenje: Port 1883 je standardni port za MQTT (nekriptirani).

```
volumes:
  - ./mqtt/config/mosquitto.conf:/mosquitto/config/mosquitto.conf
```

Objašnjenje: Mapira našu konfiguracijsku datoteku unutar kontejnera.

```
command: mosquitto -c /mosquitto/config/mosquitto.conf
```

Objašnjenje:

- `command:` - naredba koja se izvršava pri pokretanju kontejnera
- `-c` flag specificira putanju do konfiguracijske datoteke
- Zamjenjuje zadanu naredbu iz Docker slike

Servis: Backend (backend)

```
backend:
  build:
    context: .
    dockerfile: Dockerfile
```

Objašnjenje:

- `build:` - umjesto gotove slike, Docker će **izgraditi** sliku iz Dockerfile-a
- `context: .` - kontekst gradnje je trenutni direktorij (sve datoteke su dostupne)
- `dockerfile: Dockerfile` - naziv datoteke s uputama za gradnju

```
container_name: edgeai-backend
depends_on:
  db:
    condition: service_healthy
  mqtt:
    condition: service_started
```

Objašnjenje:

- `depends_on:` - definira redoslijed pokretanja servisa
- `condition: service_healthy` - backend čeka dok baza ne prođe healthcheck
- `condition: service_started` - čeka samo da se mqtt kontejner pokrene

Zašto depends_on?

Backend treba bazu i MQTT broker da bi radio. Bez `depends_on`, svi kontejneri bi se pokrenuli istovremeno i backend bi mogao pokušati spojiti se na bazu koja još nije spremna.

```
environment:
  POSTGRES_URL: "postgres://edgeai:edgeai@db:5432/edgeai?sslmode=disable"
  MQTT_BROKER: "tcp://mqtt:1883"
  SERVER_ADDR: "0.0.0.0:8080"
  DASH_USER: "admin"
  DASH_PASS: "admin123"
```

Objašnjenje:

- `POSTGRES_URL` - Connection string za bazu podataka
 - Format: `postgres://user:password@host:port/database?options`
 - `@db` - koristi ime servisa kao hostname (Docker DNS)
 - `sslmode=disable` - bez SSL enkripcije (OK za development)
- `MQTT_BROKER` - adresa MQTT brokera
 - `tcp://mqtt:1883` - TCP protokol, host je ime servisa
- `SERVER_ADDR` - adresa na kojoj backend sluša
 - `0.0.0.0` znači "sva mrežna sučelja"
- `DASH_USER/DASH_PASS` - kredencijali za pristup dashboardu

```
ports:
  - "8080:8080"
```

Objašnjenje: Izlaže backend na portu 8080.

```
restart: unless-stopped
```

Objašnjenje: Restart politika - automatski restartira kontejner osim ako nije ručno zaustavljen. Korisno za production jer se servis automatski oporavlja od pada.

Named Volume

```
volumes:
  db-data:
```

Objašnjenje: Deklarira named volume `db-data`. Docker ga kreira i upravlja njime automatski.

DATOTEKA: Dockerfile

Dockerfile sadrži **upute za izgradnju Docker slike**. Čitaju se redak po redak, od vrha prema dolje.

```
FROM golang:1.24-alpine AS builder
```

Objašnjenje:

- `FROM` - bazna slika na kojoj gradimo
- `golang:1.24-alpine` - službena Go slika temeljena na Alpine Linuxu
- `alpine` - minimalna Linux distribucija (~5MB)
- `AS builder` - imenuje ovaj stage "builder" (za multi-stage build)

Što je Multi-Stage Build?

Multi-stage build omogućuje **više FROM instrukcija** u jednom Dockerfile-u. Koristimo veliki "builder" stage za kompilaciju, a zatim kopiramo samo rezultat u malu "runtime" sliku.

Prednosti:

- Manja konačna slika (nema kompajlera, source koda)
- Bolja sigurnost (manje površine za napade)

```
WORKDIR /app
```

Objašnjenje: Postavlja radni direktorij za sve sljedeće naredbe. Ako ne postoji, kreira se automatski.

```
# GO module
COPY backend/go.mod backend/go.sum ./
```

```
RUN go mod download
```

Objašnjenje:

- `COPY backend/go.mod backend/go.sum ./` - kopira samo datoteke za dependency management
- `go mod download` - preuzima sve dependencies definirane u go.mod

Zašto prvo kopiramo samo go.mod?

Docker cachira svaki layer (korak). Ako se go.mod ne promijeni, Docker neće ponovno preuzimati dependencies pri svakoj promjeni koda. **Značajno ubrzava build.**

```
# backend kod
COPY backend/ .
```

Objašnjenje: Kopira cijeli backend direktorij u kontejner.

```
RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -o edgeai-backend
```

Objašnjenje:

- `RUN` - izvršava naredbu tijekom gradnje slike
- `CGO_ENABLED=0` - onemogućuje C-Go interop (čista Go binarka)
- `GOOS=linux` - kompilira za Linux OS
- `GOARCH=amd64` - kompilira za 64-bitnu arhitekturu
- `go build` - Go kompajler
- `-o edgeai-backend` - naziv izlazne datoteke

Cross-Compilation

Go omogućuje kompilaciju za bilo koji OS i arhitekturu s bilo kojeg računala. Možemo na Windows-u kompilirati za Linux!

Drugi Stage - Runtime

```
FROM alpine:latest
```

Objašnjenje: Počinjemo novi stage s minimalnom Alpine slikom.

```
# CA certifikati (PostgreSQL, MQTT)
RUN apk --no-cache add ca-certificates
```

Objašnjenje:

- `apk` - Alpine package manager
- `--no-cache` - ne sprema cache (manja slika)
- `ca-certificates` - SSL certifikati za sigurne konekcije

```
WORKDIR /app

COPY --from=builder /app/edgeai-backend .
```

Objašnjenje:

- `COPY --from=builder` - kopira iz prijašnjeg stage-a
- Samo kompilirana binarka, bez source koda!

```
COPY backend/templates ./templates
COPY backend/static ./static
```

Objašnjenje: Kopira HTML template i statičke datoteke (CSS, JS).

```
EXPOSE 8080
```

Objašnjenje: Dokumentira da kontejner sluša na portu 8080. Samo dokumentacija, ne otvara stvarno port.

```
CMD ["/edgeai-backend"]
```

Objašnjenje: Naredba koja se izvršava pri pokretanju kontejnera. Pokreće našu aplikaciju.

Razlika RUN vs CMD

- `RUN` - izvršava se **tijekom gradnje** slike
- `CMD` - izvršava se **pri pokretanju** kontejnera

DATOTEKA: go.mod

```
module edgeai
```

Objašnjenje: Definira naziv Go modula. Svi paketi unutar projekta su dio ovog modula.

Što je Go Module?

Go module je sustav za upravljanje dependencies u Go-u. Zamjenjuje stariji GOPATH sustav.

```
go 1.24.0
```

Objašnjenje: Minimalna verzija Go-a potrebna za ovaj projekt.

```
toolchain go1.24.11
```

Objašnjenje: Specifična verzija Go toolchaina koju koristimo.

```
require (  
    github.com/eclipse/paho.mqtt.golang v1.5.1  
    github.com/gin-gonic/gin v1.11.0  
    github.com/gorilla/websocket v1.5.3  
    github.com/jackc/pgx/v5 v5.7.6  
    golang.org/x/crypto v0.46.0  
)
```

Objašnjenje: Direktne dependencies (biblioteke koje naš kod koristi):

Biblioteka	Namjena
paho.mqtt.golang	MQTT klijent za Go
gin-gonic/gin	Web framework (HTTP server)
gorilla/websocket	WebSocket implementacija
pgx/v5	PostgreSQL driver za Go
golang.org/x/crypto	Kriptografske funkcije (bcrypt)

Ostatak `require` bloka su **indirektne dependencies** - biblioteke koje koriste naše direktne dependencies.

DATOTEKA: config.go

```
package main
```

Objašnjenje: Deklarira da ova datoteka pripada paketu `main`. U Go-u, `package main` označava izvršnu aplikaciju (ne biblioteku).

```
import (  
    "log"  
    "os"  
    "strings"  
    "golang.org/x/crypto/bcrypt"  
)
```

Objašnjenje: Importira pakete:

- `log` - jednostavno logiranje
- `os` - interakcija s operativnim sustavom (env varijable)
- `strings` - manipulacija stringova
- `bcrypt` - sigurno hashiranje lozinki

Što je Bcrypt?

Bcrypt je **algoritam za hashiranje lozinki**. Za razliku od običnih hash funkcija (MD5, SHA):

- Namjerno je spor (otežava brute-force napade)

- Automatski dodaje "sol" (salt) - čak i identične lozinke daju različite hashove
- Ima podesiv "cost factor" - možemo ga učiniti još sporijim

```
type Config struct {
    // MQTT - postavke
    MQTTBroker   string `json:"mqtt_broker,omitempty"`
    MQTTRoot     string `json:"mqtt_topic,omitempty"`
    MQTTClient   string `json:"mqtt_client,omitempty"`
    MQTTUsername string `json:"mqtt_username,omitempty"`
    MQTTPassword string `json:"mqtt_password,omitempty"`

    // Baza podataka
    PostgresURL string `json:"postgres_url,omitempty"`

    // HTTP server
    ServerAddr string `json:"server_addr,omitempty"`

    // Dashboard login
    DashboardUser   string `json:"dashboard_user,omitempty"`
    DashboardPass   string `json:"- "`
    DashboardPassHash string `json:"- "`
}
```

Objašnjenje:

- `type Config struct` - definira novu strukturu podataka (slično klasi u drugim jezicima)
- Svako polje ima:
 - **Ime** (npr. `MQTTBroker`)
 - **Tip** (`string`)
 - **Tag** (`json:"mqtt_broker,omitempty"`) - metapodaci za serijalizaciju

Struct Tags

- `json:"mqtt_broker"` - kada se struktura pretvara u JSON, koristi ovo ime
- `omitempty` - izostavi polje ako je prazno
- `json:"- "` - nikad ne uključuj u JSON (za osjetljive podatke)

```
var cfg = initConfig()
```

Objašnjenje:

- `var` - deklarira varijablu
- `cfg` - ime varijable
- `= initConfig()` - inicijalizira pozivom funkcije

Bitno: Ovo se izvršava **pri učitavanju paketa**, prije `main()` funkcije!

```
func initConfig() Config {
    c := Config{
        MQTTBroker:   getEnv("MQTT_BROKER", "tcp://localhost:1883"),
        MQTTRoot:     getEnv("MQTT_TOPIC", "edgeai/fault"),
    }
```

```
// ...  
}
```

Objašnjenje:

- `func initConfig() Config` - funkcija koja vraća `Config` strukturu
- `c := Config{...}` - kreira novu instancu `Config`-a
- `:=` - kratka deklaracija s inicijalizacijom (Go sam određuje tip)
- `getEnv("MQTT_BROKER", "tcp://localhost:1883")` - dohvati env varijablu ili koristi default

```
func getEnv(key, fallback string) string {  
    if v := os.Getenv(key); v != "" {  
        return strings.TrimSpace(v)  
    }  
    return fallback  
}
```

Objašnjenje:

- `key, fallback string` - dva parametra istog tipa
- `os.Getenv(key)` - dohvaća environment varijablu
- `strings.TrimSpace(v)` - uklanja whitespace s početka i kraja
- Ako varijabla nije postavljena ili je prazna, vraća `fallback`

```
hashFromEnv := os.Getenv("DASH_PASS_HASH")  
if hashFromEnv != "" {  
    c.DashboardPassHash = hashFromEnv  
    log.Println("CONFIG: Using pre-hashed DASH_PASS_HASH from environment")  
} else {  
    h, err := bcrypt.GenerateFromPassword([]byte(c.DashboardPass), bcrypt.DefaultCost)  
    if err != nil {  
        panic("FATAL: Failed to hash dashboard password: " + err.Error())  
    }  
    c.DashboardPassHash = string(h)  
}
```

Objašnjenje:

- Provjerava postoji li već hashirana lozinka u env
- Ako ne, hashira plaintext lozinku
- `bcrypt.GenerateFromPassword([]byte(...), bcrypt.DefaultCost)` :
 - `[]byte(...)` - pretvara string u byte slice
 - `bcrypt.DefaultCost` - broj iteracija (10)
- `panic(...)` - zaustavlja program ako hashiranje ne uspije

Error Handling u Go-u

Go nema iznimke (exceptions). Umjesto toga, funkcije vraćaju error kao drugu vrijednost:

```
value, err := someFunction()
if err != nil {
    // handle error
}
```

DATOTEKA: main.go

```
package main

import (
    "log"
    "os"
    "os/signal"
    "syscall"
    mqtt "github.com/eclipse/paho.mqtt.golang"
)
```

Objašnjenje:

- `os/signal` - za hvatanje sistemskih signala
- `syscall` - nisko-razinski sistemski pozivi
- `mqtt "github.com/..."` - alias import (koristimo `mqtt` umjesto punog imena)

```
var mqttClient mqtt.Client
```

Objašnjenje: Globalna varijabla za MQTT klijent. Potrebna jer je koriste multiple funkcije.

Globalne varijable - kada ih koristiti?

Općenito ih treba izbjegavati, ali ponekad su nužne za:

- Dijeljene resurse (baza, MQTT klijent)
- Konfiguraciju
- Loggere

```
func main() {
    InitDB()
    defer DB.Close()
}
```

Objašnjenje:

- `main()` - ulazna točka programa
- `InitDB()` - inicijalizira vezu s bazom
- `defer DB.Close()` - `defer` osigurava da se funkcija izvrši **pri izlasku** iz funkcije

Što je Defer?

`defer` je Go mehanizam za **odgođeno izvršavanje**. Koristi se za cleanup:

```
func example() {
    file := openFile()
    defer file.Close() // Izvršit će se kada funkcija završi
    // ... koristi file
} // Ovdje se poziva file.Close()
```

Defer se izvršava čak i ako funkcija završi s panik-om!

```
mqttClient = StartMQTT()
defer mqttClient.Disconnect(250)
```

Objašnjenje: Pokreće MQTT i planira disconnection pri izlasku. `250` je timeout u milisekundama.

```
StartStatusBroadcaster()
```

Objašnjenje: Pokreće goroutinu koja periodički šalje status svim klijentima.

```
router := SetupRouter()
go func() {
    log.Printf("HTTP server running on %s\n", cfg.ServerAddr)
    if err := router.Run(cfg.ServerAddr); err != nil {
        log.Fatalf("HTTP server failed:", err)
    }
}()
```

Objašnjenje:

- `SetupRouter()` - konfigurira HTTP rute
- `go func() {...}()` - pokreće **goroutinu** (konkurentnu izvršnu jedinicu)
- `router.Run(...)` - pokreće HTTP server (blokira)

Što je Goroutine?

Goroutine je **lagana nit** (thread). Go runtime upravlja tisućama goroutina efikasnije nego OS threadovima.

```
go func() {
    // Ovaj kod se izvršava paralelno
}()
```

`go` keyword pokreće funkciju u novoj goroutini. Bez `go`, `router.Run()` bi blokirao i ostatak main funkcije se nikad ne bi izvršio.

```
quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit
```

Objašnjenje:

- `make(chan os.Signal, 1)` - kreira **kanal** za signale s bufferom veličine 1

- `signal.Notify(...)` - registrira da želimo primiti SIGINT i SIGTERM
- `<-quit` - **blokira** dok ne primi signal

Što je Channel (Kanal)?

Channel je Go mehanizam za **komunikaciju između goroutina**. Siguran je za konkurentni pristup.

```
ch := make(chan int)    // Kreira kanal za int-ove

go func() {
    ch <- 42             // Šalje vrijednost u kanal
}()

value := <-ch           // Prima vrijednost iz kanala
```

Graceful Shutdown

```
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit
```

- `SIGINT` - signal koji se šalje kada pritisnete Ctrl+C
- `SIGTERM` - signal za gašenje (Docker ga šalje pri zaustavljanju)

Program čeka na jedan od ovih signala, zatim se elegantno gasi (`close()` na `faultChan`, defer funkcije se izvršavaju).

```
close(faultChan)
log.Println("Shutdown complete")
}
```

Objašnjenje:

- `close(faultChan)` - zatvara kanal, signalizira workerima da prestanu
- Defer funkcije se izvršavaju automatski (`DB.Close`, `mqttClient.Disconnect`)

DATOTEKA: db.go

```
var DB *pgxpool.Pool
```

Objašnjenje: Globalna varijabla za connection pool.

Što je Connection Pool?

Umjesto otvaranja nove konekcije za svaki upit, pool održava skup **ponovno iskoristivih konekcija**. Prednosti:

- Brže (nema overhead kreiranja konekcije)
- Kontrola resursa (maksimalan broj konekcija)
- Upravljanje životnim ciklusom

```
type FaultEvent struct {
    Timestamp time.Time `json:"timestamp"`
    Abnormal   float64  `json:"abnormal"`
    Normal     float64  `json:"normal"`
    Conclusion string   `json:"conclusion"`
}
```

Objašnjenje: Struktura koja predstavlja jedan detektirani kvar. Koristi se i za bazu i za JSON API.

```
func InitDB() {
    var err error
    DB, err = pgxpool.New(context.Background(), cfg.PostgresURL)
    if err != nil {
        log.Fatal("DB connection error:", err)
    }
}
```

Objašnjenje:

- `pgxpool.New(...)` - kreira novi connection pool
- `context.Background()` - prazan kontekst (bez timeout-a ili cancellation-a)

Što je Context?

Context je Go mehanizam za:

- Timeout/deadline (automatski cancel nakon vremena)
- Cancellation (ručni cancel)
- Proslijeđivanje podataka kroz call stack

```
if err = DB.Ping(context.Background()); err != nil {
    log.Fatal("DB ping error:", err)
}
```

Objašnjenje: Testira konekciju. Ako ne radi, program se zaustavlja.

```
func InsertFaultEvent(abnormal, normal float64) {
    if abnormal < 0 || normal < 0 {
        log.Println("Invalid values for DB insert")
        return
    }
}
```

Objašnjenje: Validacija - ne sprema negativne vrijednosti.

```
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()
```

Objašnjenje:

- `context.WithTimeout(...)` - kreira kontekst s timeout-om od 5 sekundi
- `defer cancel()` - **BITNO:** uvijek pozovi cancel da oslobodiš resurse

```
_, err := DB.Exec(ctx,
    `INSERT INTO fault_events (abnormal, normal, conclusion)
    VALUES ($1, $2, 'KVAR')`,
    abnormal, normal,
)
```

Objašnjenje:

- `DB.Exec(...)` - izvršava SQL bez vraćanja rezultata
- `$1, $2` - **parameterized query** - sprječava SQL injection
- `abnormal, normal` - vrijednosti koje zamjenjuju `$1` i `$2`

SQL Injection

Bez parametara:

```
-- Napadač unese: '; DROP TABLE users; --
query = "SELECT * FROM users WHERE name = '" + userInput + "'"
-- Rezultat: SELECT * FROM users WHERE name = '; DROP TABLE users; --
```

S parametrima:

```
-- Sigurno! Ulaz se tretira kao podatak, ne kao kod
query = "SELECT * FROM users WHERE name = $1"
```

```
func GetFaultEvents() []FaultEvent {
    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()

    rows, err := DB.Query(ctx,
        `SELECT timestamp, abnormal, normal, conclusion
        FROM fault_events
        ORDER BY timestamp DESC
        LIMIT 50`)
}
```

Objašnjenje:

- `DB.Query(...)` - vraća više redaka
- `ORDER BY timestamp DESC` - najnoviji prvi
- `LIMIT 50` - maksimalno 50 rezultata (za performance)

```
if err != nil {
    log.Println("DB query error:", err)
    return []FaultEvent{}
}
defer rows.Close()
```

Objašnjenje:

- Ako dođe do greške, vraća prazan slice (graceful degradation)

- `defer rows.Close()` - **KRITIČNO** - mora se zvati da oslobodi resurse

Graceful Degradation

Umjesto da aplikacija crashira, vraća prazan rezultat. Korisnik dobije praznu listu umjesto error stranice.

```
events := make([]FaultEvent, 0)
for rows.Next() {
    var e FaultEvent
    if err := rows.Scan(&e.Timestamp, &e.Abnormal, &e.Normal, &e.Conclusion); err != nil
{
    log.Println("DB scan error:", err)
    continue
}
    events = append(events, e)
}
return events
}
```

Objašnjenje:

- `make([]FaultEvent, 0)` - kreira prazan slice
- `rows.Next()` - pomakne se na sljedeći red, vraća `false` kad više nema
- `rows.Scan(&...)` - popunjava varijable iz trenutnog reda
- `&` - pointer (adresa varijable) - Scan treba pointer da bi mogao zapisati vrijednost
- `append(events, e)` - dodaje element na kraj slice-a

Što je Slice?

Slice je Go-ov dinamički niz. Za razliku od arraya, može rasti:

```
s := make([]int, 0)    // Prazan slice
s = append(s, 1)       // [1]
s = append(s, 2, 3)    // [1, 2, 3]
```



DATOTEKA: mqtt.go

```
var faultChan = make(chan FaultPayload, 100)
```

Objašnjenje:

- Kanal za proslijeđivanje podataka s MQTT-a na obradu
- Buffer od 100 poruka - ako se napuni, nove se preskaču

Buffered vs Unbuffered Channels

```
ch1 := make(chan int)    // Unbuffered - send blokira dok netko ne primi
ch2 := make(chan int, 10) // Buffered - može držati 10 poruka
```

```
type FaultPayload struct {
    Abnormal float64 `json:"abnormal"`
    Normal float64 `json:"normal"`
    Conclusion string `json:"conclusion"`
}
```

Objašnjenje: Struktura JSON poruke koju šalje mikrokontroler.

```
func faultWorker() {
    for payload := range faultChan {
```

Objašnjenje:

- `for ... := range faultChan` - beskonačna petlja koja prima iz kanala
- Automatski prestaje kada se kanal zatvori (`close(faultChan)`)

```
    if payload.Conclusion != "KVAR" {
        continue
    }
```

Objašnjenje: Filtrira - samo kvarove šalje dalje. `continue` preskače ostatak iteracije.

```
    InsertFaultEvent(payload.Abnormal, payload.Normal)
    event := FaultEvent{
        Timestamp: time.Now(),
        Abnormal: payload.Abnormal,
        Normal: payload.Normal,
        Conclusion: payload.Conclusion,
    }
    BroadcastFault(event)
}
```

Objašnjenje:

1. Sprema u bazu
2. Kreira `FaultEvent` s trenutnim vremenom
3. Šalje na WebSocket broadcast

```
func StartMQTT() mqtt.Client {
    go faultWorker()
```

Objašnjenje: Pokreće worker goroutinu u pozadini.

```
    opts := mqtt.NewClientOptions()
    opts.AddBroker(cfg.MQTTBroker)
    opts.SetClientID(cfg.MQTTClient)
```

Objašnjenje:

- `NewClientOptions()` - kreira options objekt
- Builder pattern - svaka metoda vraća isti objekt za chain-anje

```
if cfg.MQTTUsername != "" && cfg.MQTTPassword != "" {
    opts.SetUsername(cfg.MQTTUsername)
    opts.SetPassword(cfg.MQTTPassword)
}
client := mqtt.NewClient(opts)
```

Objašnjenje: Postavlja autentifikaciju ako je konfigurirana.

```
if token := client.Connect(); token.Wait() && token.Error() != nil {
    log.Fatal("MQTT connection error:", token.Error())
}
```

Objašnjenje:

- `client.Connect()` - vraća Token (async operacija)
- `token.Wait()` - blokira dok ne završi
- `token.Error()` - vraća grešku ako je bilo

MQTT Token Pattern

MQTT klijent koristi async operacije. Token predstavlja "obećanje" rezultata:

```
token := client.Connect()
token.Wait()           // Čekaj završetak
if token.Error() != nil {
    // Greška
}
```

```
client.Subscribe(cfg.MQTTTopic, 0, func(c mqtt.Client, msg mqtt.Message) {
    var payload FaultPayload
    if err := json.Unmarshal(msg.Payload(), &payload); err != nil {
        log.Println("MQTT JSON error:", err)
        return
    }
})
```

Objašnjenje:

- `Subscribe(topic, qos, callback)` - pretplata na topic
- QoS 0 - "fire and forget" (bez potvrde primitka)
- Callback funkcija se poziva za svaku primljenu poruku
- `json.Unmarshal(...)` - pretvara JSON u Go strukturu

MQTT QoS (Quality of Service)

- **QoS 0:** Poruka se šalje jednom, bez potvrde. Može se izgubiti.
- **QoS 1:** Poruka se šalje dok se ne potvrdi. Može doći duplicirano.

- **QoS 2:** Poruka se isporučuje točno jednom. Najsporije.

```
select {
  case faultChan <- payload:
    // Normalan flow
  default:
    log.Println("Warning: faultChan buffer full, skipped MQTT message")
}
}))
return client
}
```

Objašnjenje:

- `select` - slično switch-u, ali za kanale
- `case faultChan <- payload:` - pokušaj slanja u kanal
- `default:` - izvršava se ako kanal blokira (buffer pun)

Select Statement

Select čeka na više operacija s kanalima:

```
select {
  case msg := <-ch1:
    // Primitljeno iz ch1
  case ch2 <- value:
    // Poslano u ch2
  case <-time.After(5 * time.Second):
    // Timeout nakon 5 sekundi
  default:
    // Ako sve blokira, izvrši default
}
```

DATOTEKA: websocket.go

Što je WebSocket?

WebSocket je protokol za **dvosmjernu komunikaciju** između browsera i servera. Za razliku od HTTP-a (request-response), WebSocket održava otvorenu vezu.

HTTP:

Client: "Ima li novih podataka?" → Server: "Da, evo."

Client: "Ima li novih podataka?" → Server: "Ne."

Client: "Ima li novih podataka?" → Server: "Da, evo."

WebSocket:

Client: "Otvori vezu" → Server: "OK"

← Server: "Novi podaci!"

← Server: "Još podataka!"

← Server: "Opet novi!"

```
var upgrader = websocket.Upgrader{
    CheckOrigin: func(r *http.Request) bool {
        return true
    },
}
```

Objašnjenje:

- `Upgrader` - pretvara HTTP konekciju u WebSocket
- `CheckOrigin` - sigurnosna provjera origin-a
- `return true` - prihvaća sve origini (u produkciji provjeriti!)

CORS (Cross-Origin Resource Sharing)

Browser sprječava JavaScript da šalje zahtjeve na druge domene osim ako server to eksplicitno dozvoli.

`CheckOrigin` kontrolira tko se smije spojiti.

```
var (
    wsClients = make(map[*websocket.Conn]bool)
    wsMutex   sync.Mutex
)
```

Objašnjenje:

- `wsClients` - mapa aktivnih WebSocket konekcija
- `wsMutex` - mutex za thread-safe pristup mapi

Što je Mutex?

Mutex (Mutual Exclusion) osigurava da samo jedna goroutina pristupa resursu istovremeno:

```
var counter int
var mu sync.Mutex

func increment() {
    mu.Lock()      // Zaključaj
    counter++      // Sigurna operacija
    mu.Unlock()    // Otključaj
}
```

Bez mutex-a, dvije goroutine bi mogle čitati/pisati counter istovremeno i dobiti pogrešan rezultat.

```
func cleanupClient(conn *websocket.Conn) {
    wsMutex.Lock()
    defer wsMutex.Unlock()

    if _, ok := wsClients[conn]; ok {
        conn.Close()
        delete(wsClients, conn)
        log.Println("WebSocket client disconnected")
    }
}
```

```
}  
}
```

Objašnjenje:

- Zaključava mutex za sigurno brisanje
- `if _, ok := wsClients[conn]; ok` - provjera postoji li ključ u mapi
- `delete(wsClients, conn)` - briše iz mape

Map Lookup Pattern

```
value, ok := myMap[key]  
if ok {  
    // key postoji, value sadrži vrijednost  
} else {  
    // key ne postoji  
}
```

```
func broadcastMessage(msgType string, data interface{}) {  
    msg, err := json.Marshal(map[string]interface{}){  
        "type": msgType,  
        "data": data,  
    })
```

Objašnjenje:

- `interface{}` - Go-ov "any" tip, može držati bilo što
- `json.Marshal(...)` - pretvara Go strukturu u JSON bytes
- Kreira mapu s poljima `type` i `data`

```
wsMutex.Lock()  
defer wsMutex.Unlock()  
for conn := range wsClients {  
    if err := conn.WriteMessage(websocket.TextMessage, msg); err != nil {  
        conn.Close()  
        delete(wsClients, conn)  
    }  
}  
}
```

Objašnjenje:

- Iterira kroz sve klijente
- `WriteMessage(...)` - šalje poruku
- Ako slanje ne uspije, zatvara i briše konekciju

```
func WebSocketHandler(c *gin.Context) {  
    conn, err := upgrader.Upgrade(c.Writer, c.Request, nil)  
    if err != nil {  
        log.Println("WebSocket upgrade error:", err)
```

```
    return  
}
```

Objašnjenje:

- `Upgrade(...)` - pretvara HTTP u WebSocket
- `c.Writer, c.Request` - HTTP response writer i request iz Gin contexta

```
wsMutex.Lock()  
wsClients[conn] = true  
wsMutex.Unlock()
```

Objašnjenje: Dodaje klijenta u mapu (mutex za sigurnost).

```
defer cleanupClient(conn)
```

Objašnjenje: Osigurava cleanup kada funkcija završi.

```
conn.SetReadDeadline(time.Now().Add(60 * time.Second))  
conn.SetPongHandler(func(string) error {  
    conn.SetReadDeadline(time.Now().Add(60 * time.Second))  
    return nil  
})
```

Objašnjenje:

- `SetReadDeadline(...)` - timeout za čitanje (60 sekundi)
- `PongHandler` - kada primi "pong", resetira timeout

Ping/Pong (Heartbeat)

WebSocket koristi ping/pong za provjeru je li veza još živa:

1. Server šalje PING
2. Klijent automatski odgovara PONG
3. Ako nema PONG-a, veza je mrtva

```
pingTicker := time.NewTicker(30 * time.Second)  
defer pingTicker.Stop()
```

Objašnjenje:

- `Ticker` šalje signal svakih 30 sekundi
- `defer Stop()` - zaustavlja ticker pri izlasku

```
go func() {  
    for {  
        _, _, err := conn.ReadMessage()  
        if err != nil {  
            return  
        }  
    }  
}
```

```
    }
}
}()
```

Objašnjenje:

- Goroutina koja čita poruke
- **Mora postojati** - WebSocket zahtijeva da netko čita, inače browser prekida vezu
- Poruke se ignoriraju (samo klijent->server poruke)

```
for range pingTicker.C {
    if err := conn.WriteMessage(websocket.PingMessage, nil); err != nil {
        return
    }
}
}
```

Objašnjenje:

- pingTicker.C - kanal koji prima tick svakih 30 sekundi
- Šalje PING poruku
- Ako ne uspije, izlazi iz funkcije (cleanup se izvršava)

```
func BroadcastFault(event FaultEvent) {
    broadcastMessage("NEW_FAULT", event)
}
```

Objašnjenje: Jednostavna wrapper funkcija za broadcast kvara.

```
func BroadcastStatus() {
    status := map[string]string{
        "mqtt": "NOT CONNECTED",
        "db":    "ERROR",
    }
    if err := DB.Ping(context.Background()); err == nil {
        status["db"] = "OK"
    }
    if mqttClient != nil && mqttClient.IsConnected() {
        status["mqtt"] = "CONNECTED"
    }
    broadcastMessage("STATUS", status)
}
```

Objašnjenje: Provjerava status servisa i broadcast-a svim klijentima.

```
func StartStatusBroadcaster() {
    go func() {
        ticker := time.NewTicker(2 * time.Second)
        defer ticker.Stop()
        for range ticker.C {
```

```

        BroadcastStatus()
    }
}()
}

```

Objašnjenje: Pokreće goroutinu koja šalje status svakih 2 sekunde.

DATOTEKA: api.go

```

func BcryptAuthMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {

```

Objašnjenje:

- **Middleware** je funkcija koja se izvršava prije/poslije handlera
- `gin.HandlerFunc` - tip funkcije koju Gin očekuje
- Vraća closure (funkciju unutar funkcije)

Što je Middleware?

Middleware je kao "filter" za HTTP zahtjeve:

Request → [Auth Middleware] → [Logging Middleware] → [Handler] → Response

```

auth := c.GetHeader("Authorization")
if auth == "" || !strings.HasPrefix(auth, "Basic ") {
    c.Header("WWW-Authenticate", `Basic realm="Restricted"`)
    c.AbortWithStatus(401)
    return
}

```

Objašnjenje:

- Dohvaća Authorization header
- Provjerava počinje li s "Basic "
- Ako ne, vraća 401 Unauthorized
- `WWW-Authenticate` header govori browseru da prikaže login popup

HTTP Basic Authentication

Authorization: Basic base64(username:password)

Primjer:

username: admin

password: admin123

base64("admin:admin123") = "YWRTaW46YWRTaW4xMjM="

Header: Authorization: Basic YWRTaW46YWRTaW4xMjM=

```

payload, err := base64.StdEncoding.DecodeString(auth[len("Basic "):])
if err != nil {
    c.AbortWithStatus(401)
    return
}

```

Objašnjenje:

- `auth[len("Basic "):]` - uzima string nakon "Basic "
- `DecodeString(...)` - dekodira Base64

```

parts := strings.SplitN(string(payload), ":", 2)
if len(parts) != 2 {
    c.AbortWithStatus(401)
    return
}
username, password := parts[0], parts[1]

```

Objašnjenje:

- `SplitN(..., ":", 2)` - dijeli po ":", maksimalno 2 dijela
- Izvlači username i password

```

if username != cfg.DashboardUser {
    c.AbortWithStatus(401)
    return
}
if bcrypt.CompareHashAndPassword([]byte(cfg.DashboardPassHash), []byte(password)) !=
nil {
    c.AbortWithStatus(401)
    return
}
c.Next()
}
}

```

Objašnjenje:

- Provjerava username
- `CompareHashAndPassword(...)` - uspoređuje hash s lozinkom (sigurno, constant-time)
- `c.Next()` - nastavlja s obradom zahtjeva

```

func SetupRouter() *gin.Engine {
    r := gin.Default()
}

```

Objašnjenje:

- `gin.Default()` - kreira router s default middleware-om (logger, recovery)
- Recovery middleware hvata panic i vraća 500 umjesto da crashira server

```
r.LoadHTMLGlob("templates/*")
r.Static("/static", "./static")
```

Objašnjenje:

- `LoadHTMLGlob(...)` - učitava HTML template-e
- `Static(...)` - servira statičke datoteke

```
authorized := r.Group("/", BcryptAuthMiddleware())
authorized.GET("/", func(c *gin.Context) {
    c.HTML(http.StatusOK, "index.html", nil)
})
```

Objašnjenje:

- `r.Group(...)` - kreira grupu ruta s zajedničkim middleware-om
- Sve rute u grupi zahtijevaju autentifikaciju
- `c.HTML(...)` - renderira HTML template

```
r.GET("/api/faults", func(c *gin.Context) {
    c.JSON(http.StatusOK, GetFaultEvents())
})
r.GET("/ws", WebSocketHandler)
return r
}
```

Objašnjenje:

- `/api/faults` - REST API endpoint (bez auth - za inicijalno učitavanje)
- `/ws` - WebSocket endpoint (bez auth - za real-time)
- `c.JSON(...)` - vraća JSON response

DATOTEKA: setup.sql

```
CREATE EXTENSION IF NOT EXISTS timescaledb CASCADE;
```

Objašnjenje:

- Aktivira TimescaleDB ekstenziju
- `IF NOT EXISTS` - ne javlja grešku ako već postoji
- `CASCADE` - automatski instalira dependencies

```
CREATE TABLE IF NOT EXISTS sensor_data (
    timestamp TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    x DOUBLE PRECISION NOT NULL,
    y DOUBLE PRECISION NOT NULL,
    z DOUBLE PRECISION NOT NULL,
```

```
abnormal DOUBLE PRECISION,  
normal DOUBLE PRECISION,  
PRIMARY KEY (timestamp)  
);
```

Objašnjenje:

- `TIMESTAMPZ` - timestamp s time zone (uvijek u UTC)
- `DEFAULT NOW()` - automatski postavlja trenutno vrijeme
- `DOUBLE PRECISION` - 8-byte floating point (~15 decimala preciznosti)
- `PRIMARY KEY (timestamp)` - jedinstveni identifikator retka

Tipovi podataka

Tip	Opis
<code>TIMESTAMPZ</code>	Datum i vrijeme s time zone
<code>DOUBLE PRECISION</code>	Decimalni broj (64-bit)
<code>TEXT</code>	Tekst neograničene dužine

```
SELECT create_hypertable(  
  'sensor_data',  
  'timestamp',  
  if_not_exists => TRUE,  
  chunk_time_interval => interval '1 day'  
);
```

Objašnjenje:

- `create_hypertable(...)` - pretvara tablicu u TimescaleDB hypertable
- `'timestamp'` - kolona za particioniranje
- `chunk_time_interval => interval '1 day'` - nova particija svakog dana

Što je Hypertable?

Hypertable je TimescaleDB koncept za **automatsko particioniranje**:

- Podaci se dijele u "chunkove" po vremenu
- Svaki chunk je zasebna tablica
- Upiti na nedavne podatke su brži jer ne pretražuju stare chunkove

```
CREATE TABLE IF NOT EXISTS fault_events (  
  timestamp TIMESTAMPZ NOT NULL DEFAULT NOW(),  
  abnormal DOUBLE PRECISION NOT NULL,  
  normal DOUBLE PRECISION NOT NULL,  
  conclusion TEXT NOT NULL  
);  
  
SELECT create_hypertable(  
  'fault_events',  
  'timestamp',
```

```
    if_not_exists => TRUE
);
```

Objašnjenje: Tablica za detektirane kvarove, također hypertable.

```
CREATE INDEX IF NOT EXISTS idx_sensor_abnormal
ON sensor_data(abnormal DESC, timestamp DESC);

CREATE INDEX IF NOT EXISTS idx_faults_conclusion
ON fault_events(conclusion, timestamp DESC);
```

Objašnjenje:

- `CREATE INDEX` - kreira indeks za brže pretraživanje
- `DESC` - silazni redoslijed (novije prvo)

Što je Index?

Index je kao kazalo u knjizi - umjesto da čitaš cijelu knjigu, pogledaš kazalo i nađeš stranicu.

```
-- Bez indexa: O(n) - pregledaj svaki red
-- S indexom: O(log n) - pretraži B-tree strukturu
```

DATOTEKA: mosquito.conf

```
listener 1883 0.0.0.0
```

Objašnjenje:

- `listener` - definira gdje Mosquitto sluša
- `1883` - port broj
- `0.0.0.0` - sluša na svim mrežnim sučeljima

```
allow_anonymous true
```

Objašnjenje: Dopusća konekcije bez username/password. Za development OK, za produkciju treba promijeniti.

```
log_dest stdout
```

Objašnjenje: Logovi se ispisuju na standardni output (vidljivi u `docker logs`).

DATOTEKA: index.html

```
<!DOCTYPE html>
```

```
<html lang="hr">
```

Objašnjenje:

- `<!DOCTYPE html>` - deklarira HTML5 dokument
- `lang="hr"` - jezik stranice (hrvatski)

```
<head>  
  <meta charset="UTF-8">
```

Objašnjenje: Definira encoding (UTF-8 podržava hrvatske znakove).

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Objašnjenje: Responsive design - stranica se prilagođava veličini ekrana.

```
<title>EdgeAI - Kontrolna ploča</title>  
<meta name="description" content="...">  
<link rel="stylesheet" href="/static/style.css">  
</head>
```

Objašnjenje:

- `<title>` - naslov u tabu browsera
- `<meta description>` - opis za search engine
- `<link rel="stylesheet">` - povezuje CSS datoteku

```
<body>  
  <header class="page-header" role="banner">
```

Objašnjenje:

- `<header>` - semantički element za zaglavlje
- `role="banner"` - ARIA role za accessibility

ARIA (Accessible Rich Internet Applications)

ARIA atributi pomažu čitačima ekrana (screen readers) za slijepe korisnike da razumiju sadržaj.

```
<div class="header-title">  
  <h1>Edge AI prediktivno održavanje sustava u industriji</h1>  
  <p class="subtitle">  
    EdgeAI (nRF5340DK-TinyML) → MQTT Broker → GO Backend + TimescaleDB + WEB dashboard  
    (RPi)  
  </p>  
</div>
```

Objašnjenje: Naslov i podnaslov koji opisuju arhitekturu sustava.

```
<div id="status-bar" role="status" aria-live="polite">
  MQTT Broker: <span id="mqtt-status">Checking...</span> |
  Baza podataka: <span id="db-status">Checking...</span>
</div>
```

Objašnjenje:

- `id="mqtt-status"` - JavaScript može dohvatiti ovaj element po ID-u
- `aria-live="polite"` - obavještava screen reader o promjenama

```
<main class="dashboard-container" role="main">
  <section class="faults-panel">
    <h2>Detektirani kvarovi:</h2>
    <div class="faults-panel-content" id="sensor-data" role="log" aria-live="polite">
    </div>
  </section>
```

Objašnjenje:

- `<main>` - glavni sadržaj stranice
- `<section>` - logička sekcija
- `id="sensor-data"` - JavaScript će ovdje dodavati kvarove

```
<section class="chart-panel">
  <canvas id="faultChart" width="400" height="300">
  </canvas>
</section>
</main>
```

Objašnjenje:

- `<canvas>` - HTML element za crtanje grafike
- Chart.js koristi canvas za grafove

```
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
<script src="/static/script.js"></script>
</body>
</html>
```

Objašnjenje:

- Učitava Chart.js s CDN-a
- Učitava naš JavaScript

DATOTEKA: script.js

```
let faultChart = null;
let ws = null;
let pendingMessages = [];
let isChartReady = false;
```

Objašnjenje: Globalne varijable:

- `faultChart` - Chart.js instanca
- `ws` - WebSocket konekcija
- `pendingMessages` - buffer poruka dok chart nije spreman
- `isChartReady` - flag da li je chart inicijaliziran

```
document.addEventListener('DOMContentLoaded', () => {
  initEmptyChart();
  initWebSocket();
  loadInitial();
});
```

Objašnjenje:

- `DOMContentLoaded` - event koji se okida kad je HTML učitao
- Pokreće tri inicijalizacije

Event Listener

```
element.addEventListener('eventType', callbackFunction);
```

Browser poziva callback kada se dogodi event.

```
function initEmptyChart() {
  const ctx = document.getElementById("faultChart");

  faultChart = new Chart(ctx, {
    type: 'line',
    data: {
      labels: [],
      datasets: [{
        label: 'Abnormal',
        data: [],
        borderColor: 'red',
        pointBackgroundColor: [],
        tension: 0.3,
        fill: false
      }]
    },
    options: {
      responsive: true,
      maintainAspectRatio: false
    }
  });
};
```

```
isChartReady = true;
}
```

Objašnjenje:

- `document.getElementById(...)` - dohvaća HTML element po ID-u
- `new Chart(...)` - kreira Chart.js graf
- `type: 'line'` - linijski graf
- `labels` - X os (vremenske oznake)
- `datasets` - podaci za crtanje
- `tension: 0.3` - zakrivljenost linije
- `responsive: true` - automatski resize

```
function initWebSocket() {
  const protocol = window.location.protocol === 'https:' ? 'wss:' : 'ws:';
  ws = new WebSocket(`${protocol}://${window.location.host}/ws`);
}
```

Objašnjenje:

- Određuje protokol (ws: ili wss: za sigurnu vezu)
- `window.location.host` - hostname trenutne stranice
- Template literal (...) - string s varijablama

```
ws.onopen = () => {
  console.log("WebSocket connected to Go backend");
  flushPending();
};
```

Objašnjenje:

Callback kada se WebSocket spoji. Procesira buffered poruke.

```
ws.onmessage = (evt) => {
  try {
    const msg = JSON.parse(evt.data);
    handleIncoming(msg);
  } catch (err) {
    console.error("Invalid WebSocket JSON:", evt.data, "Error:", err);
  }
};
```

Objašnjenje:

- `onmessage` - callback za primljene poruke
- `JSON.parse(...)` - pretvara JSON string u JavaScript objekt
- `try/catch` - hvata greške parsiranja

```
ws.onclose = (event) => {
  console.warn(`WebSocket closed (code: ${event.code}), reconnecting...`);
  setTimeout(initWebSocket, 1000);
};
```

Objašnjenje:

- `onclose` - callback kada se veza prekine
- `setTimeout(initWebSocket, 1000)` - pokušava reconnect nakon 1 sekunde

Reconnect Pattern

Automatski reconnect osigurava da dashboard nastavi raditi čak i ako server na kratko nestane.

```
function handleIncoming(msg) {  
  if (!isChartReady) {  
    pendingMessages.push(msg);  
    return;  
  }  
  
  if (msg.type === 'NEW_FAULT') {  
    addFault(msg.data);  
    appendToChart(msg.data);  
  } else if (msg.type === 'STATUS') {  
    updateStatus(msg.data);  
  }  
}
```

Objašnjenje:

- Buffering dok chart nije spreman
- Routing po tipu poruke

```
async function loadInitial() {  
  try {  
    const res = await fetch('/api/faults');  
    const data = await res.json();  
  }  
}
```

Objašnjenje:

- `async/await` - moderni način pisanja asinkronog koda
- `fetch(...)` - HTTP zahtjev
- `res.json()` - parsira JSON response

Async/Await vs Callbacks

```
// Callbacks (stari način)  
fetch('/api/data')  
  .then(res => res.json())  
  .then(data => console.log(data))  
  .catch(err => console.error(err));  
  
// Async/await (moderni način)  
async function getData() {  
  try {  
    const res = await fetch('/api/data');  
    const data = await res.json();  
  }  
}
```

```

        console.log(data);
    } catch (err) {
        console.error(err);
    }
}

```

```

data.reverse().forEach(addFault);
data.forEach(appendToChart);
flushPending();

```

Objašnjenje:

- `reverse()` - obrće redoslijed (najnovije gore u listi)
- `forEach(...)` - iterira kroz array
- Zatim procesuirala buffered WS poruke

```

function appendToChart(e) {
    if (!faultChart) return;
    faultChart.data.labels.push(new Date(e.timestamp).toLocaleTimeString());
    faultChart.data.datasets[0].data.push(e.abnormal);
    faultChart.data.datasets[0].pointBackgroundColor.push(
        e.conclusion === 'KVAR' ? 'red' : 'green'
    );
}

```

Objašnjenje:

- Dodaje vremensku oznaku kao label
- Dodaje abnormal vrijednost kao data point
- Boja točke ovisno o conclusion

```

if (faultChart.data.labels.length > 50) {
    faultChart.data.labels.shift();
    faultChart.data.datasets[0].data.shift();
    faultChart.data.datasets[0].pointBackgroundColor.shift();
}
requestAnimationFrame(() => faultChart.update());
}

```

Objašnjenje:

- Ograničava na 50 točaka (performance)
- `shift()` - uklanja prvi element arraya
- `requestAnimationFrame(...)` - optimizira rendering

requestAnimationFrame

Browser optimizacija - sinkronizira update s refresh rate ekrana (obično 60fps).

```

function addFault(e) {
    const container = document.getElementById("sensor-data");

```

```

const color = e.conclusion === 'KVAR' ? 'red' : 'green';
const faultEntry = `
    <div style="background:#ffe6e6; padding:6px; margin-bottom:6px; border-left:4px
solid ${color}">
        <b>${new Date(e.timestamp).toLocaleString()}</b><br>
        Abnormal: ${e.abnormal}<br>
        Normal: ${e.normal}<br>
        Status: <span style="color:${color}; font-weight:bold">${e.conclusion}</span>
    </div>
`;
container.innerHTML = faultEntry + container.innerHTML;
}

```

Objašnjenje:

- Kreira HTML string za prikaz kvara
- `innerHTML = faultEntry + container.innerHTML` - dodaje na početak

```

function updateStatus(d) {
    const mqttEl = document.getElementById("mqtt-status");
    const dbEl = document.getElementById("db-status");
    mqttEl.textContent = d.mqtt;
    mqttEl.style.color = d.mqtt === 'CONNECTED' ? 'green' : 'red';
    dbEl.textContent = d.db;
    dbEl.style.color = d.db === 'OK' ? 'green' : 'red';
}

```

Objašnjenje: Ažurira status indikatore s odgovarajućom bojom.



DATOTEKA: style.css

```

body {
    font-family: Arial, sans-serif;
    margin: 20px;
    background: #f0f0f0;
    color: #333;
}

```

Objašnjenje:

- `font-family` - font za tekst, s fallback-om
- `margin` - razmak od rubova
- `background` - boja pozadine
- `color` - boja teksta

```

.dashboard-container {
    display: flex;
    gap: 20px;
}

```

```
    align-items: stretch;
    min-height: 400px;
}
```

Objašnjenje:

- `display: flex` - Flexbox layout
- `gap` - razmak između elemenata
- `align-items: stretch` - elementi se razvlače na istu visinu

Flexbox

Flexbox je CSS layout model za 1D rasporede (redak ili stupac):

```
.container {
    display: flex;
    flex-direction: row;    /* ili column */
    justify-content: center; /* horizontalno poravnanje */
    align-items: center;    /* vertikalno poravnanje */
}
```

```
.faults-panel,
.chart-panel {
    flex: 1;
    min-height: 400px;
    max-height: 400px;
    padding: 8px;
    border-radius: 8px;
    border: 1px solid #ccc;
    display: flex;
    flex-direction: column;
}
```

Objašnjenje:

- `flex: 1` - oba panela dijele prostor jednako
- `flex-direction: column` - unutarnji elementi su u stupcu

```
.faults-panel-content {
    flex: 1;
    overflow-y: auto;
    min-height: 0;
}
```

Objašnjenje:

- `overflow-y: auto` - scroll ako sadržaj prelazi
 - `min-height: 0` - fix za flex overflow
-

DATOTEKA: package.json

```
{
  "dependencies": {
    "chart.js": "^4.5.1"
  }
}
```

Objašnjenje:

- package.json je konfiguracijska datoteka za Node.js projekte
- U ovom slučaju samo dokumentira da projekt koristi Chart.js
- ^4.5.1 - verzija 4.5.1 ili novija patch/minor verzija

Napomena: Chart.js se zapravo učitava s CDN-a u HTML-u, ne instalira se lokalno.

TOK PODATAKA (DATA FLOW)

1. Senzor šalje podatke

nRF5340DK mikrokontroler:

1. Čita vibracije s akcelerometra
2. TinyML model klasificira: "OK" ili "KVAR"
3. Šalje JSON na MQTT topic: edgeai/fault

2. MQTT prima i prosljeđuje

Mosquitto broker:

1. Prima poruku na port 1883
2. Prosljeđuje svim subscriberima (Go backend)

3. Go backend obrađuje

StartMQTT() → Subscribe callback:

1. Prima JSON poruku
2. Parsira u FaultPayload strukturu
3. Šalje u faultChan kanal

faultWorker() goroutina:

1. Prima iz faultChan
2. Filtrira (samo KVAR)
3. InsertFaultEvent() → sprema u TimescaleDB
4. BroadcastFault() → šalje na WebSocket

4. Frontend prima i prikazuje

```
WebSocket onmessage:
```

1. Prima JSON poruku
2. `handleIncoming()` routira po tipu
3. `NEW_FAULT` → `addFault()` + `appendToChart()`
4. `STATUS` → `updateStatus()`



ARHITEKTURALNI KONCEPTI

Microservices Architecture

Projekt koristi mikroservisnu arhitekturu:

- **db** - servis za bazu
- **mqtt** - servis za messaging
- **backend** - aplikacijska logika

Prednosti:

- Nezavisno skaliranje
- Različite tehnologije za svaki servis
- Izolacija grešaka

Event-Driven Architecture

Sustav je vođen događajima:

1. Senzor **objavljuje** događaj (MQTT publish)
2. Backend **reagira** na događaj
3. Dashboard **prima** event kroz WebSocket

Producer-Consumer Pattern

```
Producer (MQTT callback) → Buffer (faultChan) → Consumer (faultWorker)
```

Buffer odvaja brzog producera od sporijeg consumera.

Pub/Sub Pattern

MQTT koristi publish/subscribe:

- Publisheri ne znaju tko sluša
- Subscriberi ne znaju tko šalje
- Broker spaja sve



SIGURNOSNI KONCEPTI

Password Hashing (Bcrypt)

Lozinke se nikad ne spremaju u plain textu:

```
plaintext: "admin123"
hash: "$2a$10$N9qo8uLOickgx2ZMRZoMye..."
```

- Sporo (otežava brute-force)
- Salt (identične lozinke daju različite hashove)

SQL Injection Prevention

Korištenje parameterized queries:

```
// OPASNO
query := "SELECT * FROM users WHERE name = '" + userInput + "'"

// SIGURNO
DB.Query("SELECT * FROM users WHERE name = $1", userInput)
```

Basic Authentication

HTTP Basic Auth šalje credentials u svakom zahtjevu:

- Base64 encoded (NE enkripcija!)
- Treba HTTPS u produkciji

KAKO POKRENUTI PROJEKT

```
# 1. Kloniraj projekt
git clone <repo-url>
cd EdgeAI_RPi_Docker

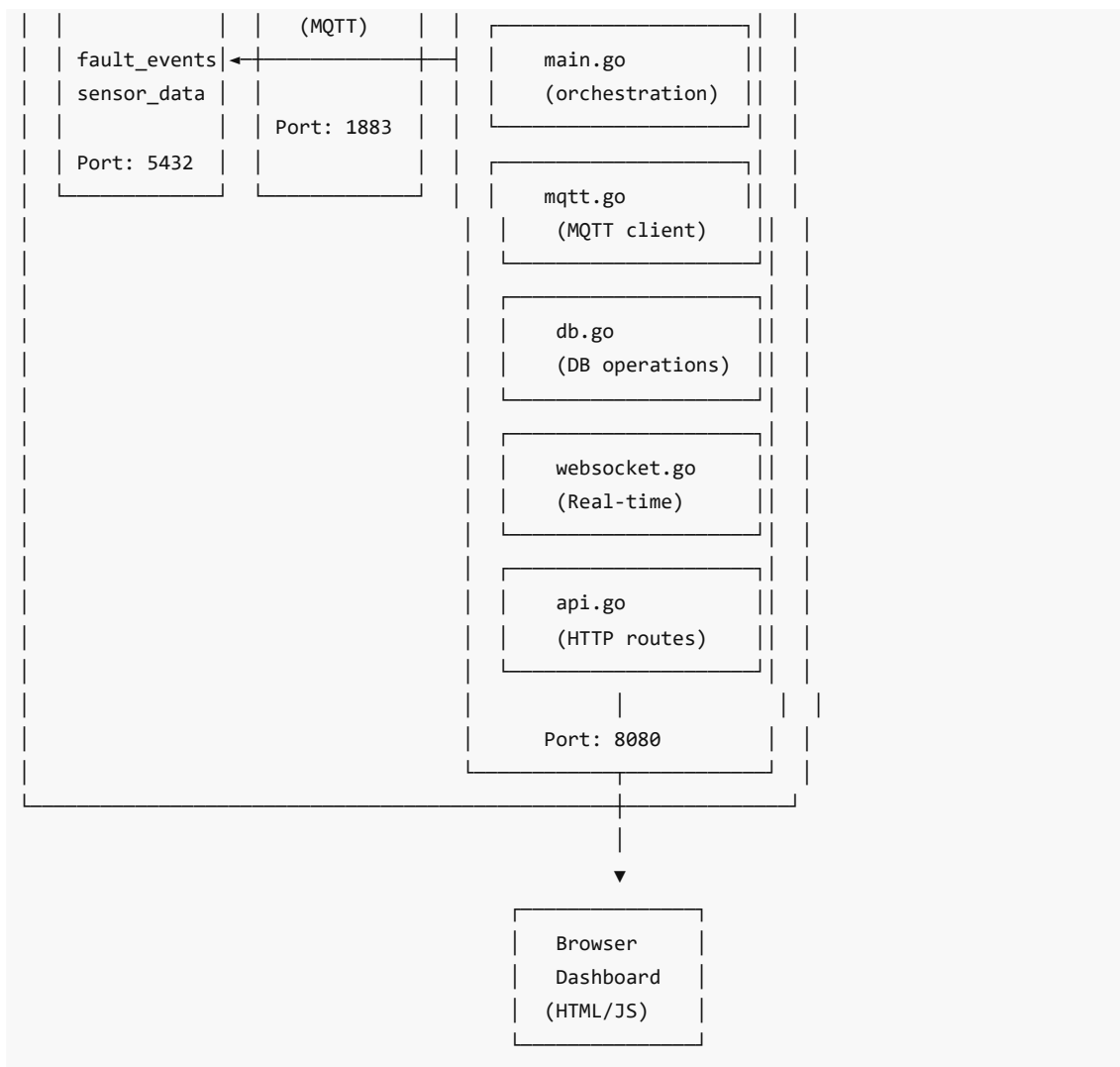
# 2. Pokreni sve servise
docker-compose up -d

# 3. Provjeri logove
docker-compose logs -f backend

# 4. Otvori dashboard
# http://localhost:8080
# Username: admin
# Password: admin123
```

DIJAGRAM KOMPONENATA





SAŽETAK KLJUČNIH KONCEPATA

Koncept	Objašnjenje
Docker	Kontejnerizacija aplikacija
Docker Compose	Orkestracija više kontejnera
Go	Programski jezik za backend
Goroutine	Lagana konkurentna nit
Channel	Komunikacija između goroutina
MQTT	Lagani messaging protokol za IoT
Pub/Sub	Publisher/Subscriber pattern
TimescaleDB	Baza za vremenske serije

Hypertable	Automatsko particioniranje
WebSocket	Dvosmjerna real-time komunikacija
REST API	HTTP endpoint za podatke
Gin	Go web framework
Basic Auth	HTTP autentifikacija
Bcrypt	Sigurno hashiranje lozinki
Connection Pool	Ponovno korištenje DB konekcija
Context	Timeout i cancellation
Middleware	Filter za HTTP zahtjeve
Chart.js	JavaScript biblioteka za grafove
Flexbox	CSS layout model
Multi-stage Build	Dockerfile optimizacija

Ovaj dokument detaljno objašnjava svaku liniju koda i sve koncepte potrebne za razumijevanje EdgeAI projekta.